GridSearchCV vs. RandomizedSearchCV

1. Introduction

Hyperparameter tuning is crucial for optimizing machine learning models. Two popular techniques in **Scikit-Learn** are:

- **GridSearchCV** (Exhaustive Search)
- **RandomizedSearchCV** (Randomized Search)

2. GridSearchCV (Grid Search Cross-Validation)

GridSearchCV**exhaustively** searches over all possible hyperparameter combinations provided in a predefined grid.

Example

If tuning **C** and **gamma** for an SVM:

C	Gamma
0.1	0.001
0.1	0.01
0.1	0.1
1	0.001
1	0.01
1	0.1

GridSearchCV evaluates **all** possible combinations.

Python Implementation

```
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC

param_grid = {'C': [0.1, 1, 10], 'gamma': [0.001, 0.01, 0.1]}

grid_search = GridSearchCV(SVC(), param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train, y_train)

print("Best Parameters:", grid_search.best_params_)
print("Best Score:", grid_search.best_score_)
```

✅ Pros:

- Guarantees finding the best combination.
- Simple to implement.

❌ Cons:

- Computationally expensive for large hyperparameter spaces.

3. RandomizedSearchCV (Randomized Search Cross-Validation)

RandomizedSearchCV**randomly samples** hyperparameter combinations instead of trying all possibilities.

Example

For the same **C** and **gamma** grid:
Instead of testing **9** combinations, it randomly selects **N** combinations (e.g., **4**).

Python Implementation

```
from sklearn.model_selection import RandomizedSearchCV
from sklearn.svm import SVC
import numpy as np

param_dist = {'C': np.logspace(-3, 3, 10), 'gamma': np.logspace(-3, 3, 10)}

random_search = RandomizedSearchCV(SVC(), param_dist, n_iter=5, cv=5, scoring='accuracy', random_state=42, n_jobs=-1)
random_search.fit(X_train, y_train)

print("Best Parameters:", random_search.best_params_)
print("Best Score:", random_search.best_score_)
```

✅ **Pros:**

- Faster than GridSearchCV.

- Useful for high-dimensional search spaces.

❌ **Cons:**

- Might miss the best hyperparameter combination.

4. Key Differences: GridSearchCV vs. RandomizedSearchCV

Feature	GridSearchCV	RandomizedSearchCV
Search Method	Exhaustive	Random Sampling
Time Complexity	High (for large spaces)	Lower (faster)
Best for	Small search spaces	Large search spaces
Guarantees Best Parameters?	✅ Yes	❌ No (Approximate)
Computational Cost	Expensive	More efficient

5. When to Use What?

- **Use GridSearchCV** when:
  - The search space is **small**.
  - Computational resources are **not a constraint**.
  - You need the **absolute best parameters**.
- **Use RandomizedSearchCV** when:
  - The search space is **large**.
  - You want a **quick approximation** of the best parameters.
  - Computational efficiency is a priority.

6. Interview Questions & Answers

Q1: What is the difference between GridSearchCV and RandomizedSearchCV?

A:GridSearchCV tries **all** combinations, while RandomizedSearchCV**randomly selects** a subset.

Q2: Which method is better for high-dimensional hyperparameter tuning?

A:RandomizedSearchCV is better as it avoids exhaustive search and speeds up computation.

Q3: Does RandomizedSearchCV always find the best parameters?

A: No, since it selects **random combinations**, it might miss the optimal one.

Q4: When should we use GridSearchCV?

A: When the search space is **small** and computational cost is not a concern.

7. Important Links for Further Learning

📎 **Scikit-learn GridSearchCV Documentation**: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

📎 **Scikit-learn RandomizedSearchCV Documentation**: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html

📎 **Machine Learning Mastery – Hyperparameter Tuning**: https://machinelearningmastery.com/hyperparameter-optimization-with-random-search-and-grid-search/